

Chapter 15. Modern Performance and The Future

In this chapter, we will look to the future of performance for Java and the JVM, especially as it relates to the reality of cloud native deployments. We will discuss several technologies that are not present (or present only as incubator or preview features) in Java 21. This reflects the principle that we only cover final features present in an LTS version in the main part of the book, and we reserve discussion of non-final features until this chapter.

New Concurrency Patterns

In this section, we're going to discuss some new patterns for concurrent systems that are enabled by virtual threads and some related new features that “follow on” from virtual threads—specifically *structured concurrency* (JEP 453) and *scoped values* (JEP 446).

Note that, as of JDK 21, both structured concurrency and scoped values are in preview state, so they cannot be used in production applications.

Structured Concurrency

The first of the two new APIs is known as *structured concurrency*. This is an API for thread handling, which provides an approach for cooperating tasks (usually virtual threads) to be considered and managed collectively as a collection of subtasks.

It might help to recall the discussion of Amdahl's law in [Chapter 13](#), where we described the application of concurrent techniques to data-parallel problems.

In contrast, structured concurrency is designed for task-parallel problems and, because of its affinity to virtual threads, it is primarily useful for tasks that involve some amount of I/O (especially calls to remote services). However, the approach is much less useful for operations that act solely (or mostly) on in-memory data, as the virtual threads will contend with each other for CPU time.

The general flow for a structured concurrency task looks something like this:

1. Create a scope—the creating thread *owns* the scope. The scope enables the grouping of subtasks to coordinate the tasks in the group.
2. Fork concurrent subtasks in the scope (each is a virtual thread).
3. Scope owner joins the scope (all subtasks) as a unit.
4. Scope's `join()` method blocks until all subtasks have completed.
5. After joining, handle any errors in forks and process results.
6. Close the scope.

It's worth pointing out that the version of structured concurrency that shipped in Java 21 included some minor API changes over Java 20. The main one is that `fork()` now returns a `Subtask` (which implements `Supplier`) instead of a bare `Future` (as it was in Java 20).

TIP

Java's release cadence and preview APIs are key in providing early access for real-world feedback.

The reason for this new interface, rather than just using `Future`, is that results are queried only after a `join()` because structured concurrency treats multiple subtasks as single unit of work. As a result, neither blocking calls to `get()` nor checked exceptions from subtasks are useful, so `Future` was something of an awkward interface; `Subtask` is a checked-exception-free interface.

Let's see structured concurrency in action in an example using the calculation of a stock tip, a record class that we'll define like this:

```
record StockTip(String symbol, double sentiment, double delta24) {}
```

We'll assume that the strength of the market's attitude to the stock (the *sentiment*) and the possible change in price over the next 24 hours (the *delta24*) are to be calculated by some external process. These elements may take some time to compute, and this is likely to involve network traffic.

We can therefore use structured subtasks to compute them, like this:

```
String symbol = "IBM";

try (var scope = new StructuredTaskScope.ShutdownOnFailure()) {
    Callable<Double> getSentiment = () -> getSentiment(symbol);
```

```

        Subtask<Double> fSentiment = scope.fork(getSentiment());

        Callable<Double> getDelta = () -> getDelta24(symbol);
        Subtask<Double> fDelta = scope.fork(getDelta);

        scope.join();
        scope.throwIfFailed();

        return new StockTip(symbol, fSentiment.get(), fDelta.get());
    } catch (ExecutionException | InterruptedException e) {
        throw new RuntimeException(e);
    }
}

```

This follows the general flow for structured concurrency that we established previously.

Closing the scope is handled implicitly via the try-with-resources block—this shuts down the scope and waits for any straggling subtasks to complete. `StructuredTaskScope` has different Shutdown policies. In the previous example, we used `ShutdownOnFailure()`; in the next example, we will use `ShutdownOnSuccess()` in try-with-resources.

We should also mention a couple of other points.

First, joining the subtasks can also be canceled by calling a `shutdown()` method. Second, there is also a timed variant of `join()`, called `joinUntil()`, which accepts a deadline (as an `Instant` parameter).

There are two built-in shutdown policies for the scope (and custom shutdown policies are also supported):

- Cancel all subtasks if one of them fails (`ShutdownOnFailure`).
- Cancel all subtasks if one of them succeeds (`ShutdownOnSuccess`).

We met the first of these built-in options in our first example, so let's move on to meet the second option.

Consider a library method where multiple subtasks are launched (possibly multiple copies of the same subtask), and the first result (from any of the subtasks) will do. The tasks are racing each other to complete, and the rest of the virtual threads should be shut down as soon as the first success occurs, so we should use the `ShutdownOnSuccess` policy, like this:

```

<T> T race(List<Callable<T>> tasks, Instant deadline)
    throws InterruptedException, ExecutionException, TimeoutException

    try (var scope = new StructuredTaskScope.ShutdownOnSuccess<T>()) {
        for (var task : tasks) {

```

```

        scope.fork(task);
    }
    return scope.joinUntil(deadline)
        .result(); // Throw if none of the subtasks
                  // completed successfully
    }
}

```

This has an obvious dual operation: all tasks must run to completion, and a failure of any subtask should cancel the entire task. To achieve this, we'll use `ShutdownOnFailure` again:

```

<T> List<T> runAll(List<Callable<T>> tasks)
    throws InterruptedException, ExecutionException {

    try (var scope = new StructuredTaskScope.ShutdownOnFailure()) {
        List<? extends Subtask<T>> handles =
            tasks.stream().map(scope::fork).toList();

        scope.join()
            .throwIfFailed(); // Propagate exception if any subtask fail

        // Here, all tasks have succeeded, so compose their results
        return handles.stream().map(Subtask::get).toList();
    }
}

```

Note that this version of the code rematerializes the results into a `List`, but it's also possible to imagine a version that had a different terminal operation, which reduced the results and returned a single value.

We can build more complex structures as well—the subtasks we created using forks can themselves create scopes (subscopes). This naturally induces a tree structure of scopes and subtasks, which is useful when we want to condense a final value out of a tree of subtasks.

If, however, the main point of our code is to operate via side effects, then it is possible to use a `StructuredTaskScope<Void>` —i.e., use a task scope that returns `void`, such as in this example:

```

void serveScope(ServerSocket serverSocket) throws IOException,
    InterruptedException {
    try (var scope = new StructuredTaskScope<Void>()) {
        try {
            while (true) {
                final var socket = serverSocket.accept();
                Callable<Void> task = () -> {
                    handle(socket);
                };
            }
        }
    }
}

```

```

        return null;
    };
    scope.fork(task);
}
} finally {
    // If there's been an error or we're interrupted,
    // we stop accepting
    scope.shutdown(); // Close all active connections
    scope.join();
}
}
}

```

However, this is arguably often better handled using a fire-and-forget pattern, such as `newVirtualThreadPerTaskExecutor()`. There are also some small wrinkles with the generics here—such as needing to explicitly `return null`.

One recurring theme in all the patterns that we have met so far is that using these techniques requires applying design thinking and knowledge of the domain and context of the problem being solved. There is no software tool that can tell with 100% accuracy whether a thread is a good candidate for being converted to a vthread—that is a task for a human software engineer.

Likewise, the restructuring of a task into subtasks and the definition of the relevant scopes requires the programmer to have a good understanding of the domain and any data dependencies between the subtasks.

Let's move on to look at the second of the new APIs that we want to discuss.

Scoped Values

As well as structured concurrency, the new *Scoped Values* API arrived in Java 21 as a preview. It is based on a new class, `ScopedValue<T>` in `java.lang`, and it represents a *binding* of a value to a variable within a specific scope. This value is written once and is then immutable on a per-scope basis.

The scope-specific bound value can be retrieved at any point down any call chain within the scope, but only within the scope in which it was set—this provides robustness and a form of encapsulation.

In particular, there is no need to explicitly pass the scoped value down the call chain. It can be thought of as *implicitly* available, but this is a

much more controlled (and more Javaish) form than, say, Scala's implicit method parameters.

The Scoped Values API can also be thought of as a modern alternative to thread-local variables but with a number of enhancements, such as immutability. This means there is no `set()` method to let faraway code change a scoped value. This also enables possible future runtime optimizations, as the runtime can be certain that a scoped value cannot change.

Some goals of the API are:

- To share data within a thread and with child threads.
- Controlled and bounded lifetime of values.
- Lifetimes visible from the structure of code.
- Immutability allows sharing by lots of threads.
- Immutability and explicit lifetime is often a better fit.

It is not necessary for programmers to move away from `ThreadLocal`, but scoped values combine well with virtual thread patterns, such as fire-and-forget. It therefore seems quite likely that as scoped values are adopted, then `ThreadLocal` will be gradually replaced for almost all use cases.

Let's rewrite the virtual thread web server to use scoped values:

```
public class ServerSV {
    private final static ScopedValue<Socket> SOCKETS_V =
        ScopedValue.newInstance();

    void serve(ServerSocket serverSocket) throws IOException,
        InterruptedException {
        while (true) {
            var socket = serverSocket.accept();
            ScopedValue.where(SOCKETS_V, socket)
                .run(() -> handle());
        }
    }

    private void handle() {
        var socket = SOCKETS_V.get();
        // handle incoming traffic
    }
}
```

Note that the `handle()` method now no longer takes a parameter; instead, the socket is accessed via the scoped value—this is the implicit availability we discussed previously. This example is very simple, as all

we’re really doing is replacing the parameter passing with a scoped value—an almost trivial application. `ScopedValue.where` presents a scoped value and the object it is to be bound to. On the execution of `run`, the value is bound, which provides a copy specific to the current thread. Calling `.get()` reads the scoped value, and on completion of the method, the binding is destroyed.

The real power of scoped values is that the call chains and the scoping and subsampling can be arbitrarily complex, and the scoped value will still be available.

Overall, the intent of scoped values is to provide a *dynamic scope*, a concept that has not been seen in Java before. This approach to scopes is similar to that found in some other languages—such as shells, Lisp dialects, and Perl. It’s also important to notice that the creation of the `private final static` field happens in object context (as the class is loaded), but the dynamic scope must be created within a method.

We can contrast it with the traditional Java form of scoping—usually called *lexical scoping*. This is where the scope of a variable is determined by the structure of the code, usually defined by a matching pair of curly braces.

Our dynamic scoping example shows a key pattern in action:

- Using a `static final` field as a holder for a scoped value
- Declaring the `ScopedValue` instance in class scope
- Creating the dynamic scope (e.g., `runWhere()`) within a method
- Using a lambda to define the scope body (where the call chains will live)

Scoped values are intended to be very useful for passing values like transaction contexts and other examples of *ambient context* data.

Scoped values interact well with structured concurrency, as they can be constructed for a scope and then *rebound* by subsamples. Any values that are not rebound will be inherited by the subsample. This technique allows for “privilege escalation” and similar patterns, such as in this example, where we will consider two security access levels:

```
enum SecurityLevel { USER, ADMIN }
```

We’ll use a scoped value to hold the current security level and another to hold the current request number:

```

private static final ScopedValue<SecurityLevel> securitySV =
    ScopedValue.newInstance();
private static final ScopedValue<Integer> requestSV = ScopedValue.newInsta

private final AtomicInteger req = new AtomicInteger();

public void run() {
    //Present the binding of the current security level
    ScopedValue.where(securitySV, level())
        //Present the binding the current request number
        .where(requestSV, req.getAndIncrement())
        //Bind the values
        .run(() -> process());
}

```

To demonstrate rebinding, let us assume that ADMIN privileges are not available, so any attempt to use them will result in a fallback to user privileges:

```

private void process() {
    if (!securitySV.isBound()) {
        throw new RuntimeException(
            "ScopedValue not bound - this should not happen");
    }

    var level = securitySV.get();
    if (level == SecurityLevel.USER) {
        System.out.println("User privileges granted for " +
            requestSV.get() + " on: " + Thread.currentThread());
    } else {
        //ADMIN is not available in our implementation
        System.out.println("Admin privileges requested for " +
            requestSV.get() + " on: " + Thread.currentThread());
        System.out.println(
            "System is in lockdown. Falling back to user privileges");
        //Present and bind the USER level and execute process
        //again with the new security level
        ScopedValue.where(securitySV, SecurityLevel.USER)
            .run(() -> process());
    }
}
}

```

To conclude this section, we should also point out that classes that represent continuations and other low-level building blocks for virtual threads and other components do exist in Java 21. However, they are in the package `jdk.internal.vm`, so they are not intended for direct use by Java programmers as of this release.

We can expect both of these APIs to continue to be developed, and hopefully arrive in a final form in some future version of Java. Let's move on to look at some of the major OpenJDK projects being developed over the last few years.

Panama

Project Panama is a major new OpenJDK project that gets its name from the *Panama Canal*, which connects the Atlantic and Pacific Oceans. In Project Panama's case, it connects the JVM and native code.

...[I]mproving and enriching the connections between the Java virtual machine and well-defined but “foreign” (non-Java) APIs, including many interfaces commonly used by C programmers.

—[Project Panama](#)

It comprises JEPs in two main areas:

- Foreign Function and Memory API
- Vector API

The Foreign Function and Memory (FFM) API was originally proposed as a preview feature in Java 19 and subsequently refined in Java 20 and Java 21, before being finalized in Java 22.

However, because Java 22 is not an LTS release, we choose to cover Panama here rather than earlier, as there is no current LTS that contains the API as a final feature. As of Java 21, the API lives in the `jdk.incubator.foreign` package in the `jdk.incubator.foreign` module, and in the Java 22 final feature in the package `java.lang.foreign` in the `java.base` module.

Panama provides direct support in Java for:

- Foreign memory allocation
- Manipulation of structured foreign memory
- Lifecycle management of foreign resources
- Calling foreign functions

The implementation builds upon the `MethodHandles` and `VarHandles` APIs, and its overall design goals are:

Productivity

Replace the brittle machinery of native methods and the Java Native Interface (JNI) with a concise, readable, and pure-Java API.

Performance

Provide access to foreign functions and memory with overhead comparable to, if not better than, JNI and `sun.misc.Unsafe`.

Broad platform support

Enable the discovery and invocation of native libraries on every platform where the JVM runs.

Uniformity

Provide ways to operate on structured and unstructured data, of unlimited size, in multiple kinds of memory (e.g., native memory, persistent memory, and managed heap memory).

Soundness

Guarantee no use-after-free bugs, even when memory is allocated and deallocated across multiple threads.

Integrity

Allow programs to perform unsafe operations with native code and data but warn users about such operations by default.

Two of the most important concepts in Panama are the *arena* and the *memory segment*. A simple demonstration of them can be seen in this example:

```
public class Main {
    private static final int INT_SIZE = 4;
    private static final long ARENA_SIZE = 4 * 1024 * 1024 * 1024L;

    public static void main(String[] args) {
        long l = 0;
        try (var arena = Arena.ofConfined()) {
            MemorySegment segment = arena.allocate(INT_SIZE * ARENA_SIZE);
            for (l = 0 ; l < ARENA_SIZE ; l += 1) {
                segment.setAtIndex(ValueLayout.JAVA_INT, l, (int)(l % 16))
            }
        }
        System.out.println("l = " + l);
    }
}
```

There are several things to note:

- The `Arena` class is used to control the lifecycle of memory segments.
- Arenas use the familiar try-with-resources construct to guarantee deterministic deallocation (which may need to be coordinated across segments).
- Memory segments are allocated from the arena.
- The `allocate()` method takes a `long` argument, allowing larger chunks of memory to be allocated than allowed by the `ByteBuffer` class (or arrays).

In this example, we are using a *confined arena*—this is the simplest case, as it represents an arena that can only be used by the current thread. Panama also supports shared arenas, a global arena, and also an automatic arena (which is managed by the JVM’s GC).

You may have noticed that we have discussed the Foreign Memory API but have not mentioned the Vector API. This is because the Vector API has made the decision to incubate until certain necessary features of Project Valhalla (see [“Valhalla”](#)) become available as preview features.

This places the Vector API farther out, and on a much more speculative basis, than some of the other features we are discussing in this chapter.

So, instead, let’s take a look at a new OpenJDK project that is relevant to the discussion of evolving Java execution, but that may need to be coordinated, and is still in its very early stages at the time of writing (August 2024).

Leyden

Project Leyden is named for *Leyden jars*, which were an early form of electrical capacitor dating from the 18th century and invented in the city of Leyden in the Netherlands.

NOTE

Another name for a capacitor is a *condenser*, which has some significance in terms of naming aspects of the project, as we will see later.

The overall aim of the project is:

To improve the startup time, time to peak performance, and footprint of Java programs.

Colloquially, the name is intended to invoke “capturing lightning in a bottle”—i.e., preserving the semantics of Java programs without requiring the overhead of the general-purpose dynamic capabilities that HotSpot provides.

This is rooted in the idea that the JVM balances both static and dynamic reasoning about runtime states and optimization, rather than the “choose one, lose one” approach taken by other languages.

For example, languages like C++ choose static reasoning and compilation, and give up dynamism, whereas languages like Python choose dynamic reasoning and then struggle to add back limited forms of static reasoning.

In contrast, HotSpot speculatively optimizes dynamic states at runtime, in effect converting them to static states. In Leyden, the goal is that such optimizations can be shifted and speculatively optimized before application startup.

Note that this is more general than just “provide AOT compilation.” As we discussed in [Chapter 6](#), there is a distinction between an outcome and a mechanism—and Leyden is focused on outcomes.

Leyden draws upon the practical experience that has already been gained by projects such as GraalVM and Quarkus, and it seeks to generalize this experience and bring it into the core of OpenJDK and the Java standards.

The two fundamental mechanisms being explored in Leyden currently are:

- Condensers
- Premain archives

Let’s look at each in turn.

Images, Constraints, and Condensers

One of the most important ideas within the project is that of a *static runtime image*. This is understood to be a standalone program, derived from an application and a JDK, which runs solely on that specific application.

A related concept is that of the *closed world* constraint. An application that signs up to this constraint indicates that it is prepared to accept some strict limitations on classes that it can load: during the runtime phase, it cannot load classes from outside the image, nor can it create classes dynamically.

The closed-world constraint imposes very strict limits on Java’s natural dynamism, particularly on the run-time reflection and class-loading features. However, so many of Java’s existing libraries and frameworks depend upon these aspects, and as a result, not all applications are well suited to this constraint, and not all developers are willing to live with it.

Therefore, rather than adopt the closed-world constraint as a primary and singular goal, Leyden instead pursues a gradual and incremental approach—it seeks to explore what intermediate states exist. This is expressed in terms of constraints weaker than closed world that are still useful and appropriate for a meaningful number of Java workloads.

Noting that Java and the JVM—by design—have dynamic features that make static analysis difficult (or even impossible), Leyden’s approach gives developers the control to trade functionality for performance—and to do so selectively.

One of the key concepts is *computation shifting*—moving certain types of computation out of the startup and warmup phases of an application into earlier (or in some cases later) phases.

We can shift two kinds of computation:

- Work expressed directly by a program (e.g., invoke a method)
- Work done on behalf of a program (e.g., compile a method to native code)

Java implementations already have some features that can shift computation automatically:

- Compile-time constant folding (shifts computation earlier)
- Pre-digested class-data archives (earlier)
- Lazy class loading and initialization (later)

Both Quarkus’s build time computation and the AOT compilation capabilities of GraalVM can be seen as shifting compilation earlier (although these capabilities are tied to the framework and are not standardized).

NOTE

From a certain point of view, even garbage collection can be seen as shifting computation to later phases.

Whenever shifting occurs, it must always preserve program meaning, per the Java specifications, this is necessary to ensure compatibility. Leyden will explore new ways to shift computation.

Some kinds of shifting will likely require no specification changes, but some of the possibilities being considered definitely will, and the intent is also to provide new features that allow developers to express their intent to shift computation directly.

A *condenser* is a transformation that is intended to shift computation from runtime to earlier phases by examining the entire program image—i.e., it is a *meaning-preserving whole-program transformation*.

Condensers will transform a program image into a new image that may contain:

- New code (AOT compiled methods)
- New data (serialized heap objects)
- New metadata (such as preloaded classes)
- New constraints

Note that condensers are intended to be composable—the image output by one condenser can be the input to another, and a particular condenser can be applied multiple times, if necessary.

NOTE

Experience with Quarkus Native Mode suggests that when unit testing or debugging, don't bother performing the program transformations—just run normally. Performing this type of test would be a net result in testing the framework and not provide value.

Shifting computation generally requires accepting constraints, with the overall idea that you can trade functionality for performance via the condensers that you choose.

Given sufficiently powerful condensers, if you shift enough computation earlier or later in time, you might even be able to produce a fully static native image, although this will likely require accepting many constraints.

This means that Leyden need not necessarily specify fully static native images directly. Instead, it will enable sufficient shifting of computation and constraining of dynamism, so that fully static native images can fall out as an emergent property.

At the time of writing (August 2024), a number of design efforts are underway, but not much work toward condensers has landed in mainline yet.

This aspect of the project is still early in its development and has commenced by looking at such ideas as resolving `invokedynamic` linkages at compile time, where possible (e.g., for lambdas), and the development of lazily computed static final fields.

Let's move on to look at the other major aspect of Leyden—the premain.

Leyden Premain

The aim of Leyden premain is to reduce warmup activity—which we define as optimization effort (by the JVM, not app) to reach peak performance. Peak performance may be defined as a statistical maximum (with some noise still present).

As the JVM is usually quite a noisy environment, with noise often in the 3%-5% range, then we can define a rule that peak is reached at 95% throughput or better. The warmup time is, therefore, defined to be the time it takes to reach 95% throughput.

To achieve this, the premain aspect of Leyden builds on the concept of *class-data sharing* (CDS).

This is not a new idea—CDS has been available in Java since version 8 and has been part of the default installation starting (for the LTS versions) with Java 17. The basic idea is that when the JVM starts, a shared archive is memory-mapped in to allow immediate availability of read-only JVM metadata for a selection of classes, thus shortening startup time.

In current versions of Java, by default, those classes come from the standard Java library. However, recent enhancements also allow for application class-data sharing (AppCDS), which are more flexible and under the control of the developer. They have been introduced to extend the CDS concept to include selected classes from the application class path.

This even includes the ability (as of Java 17) to produce *dynamic AppCDS archives*, whereby metadata can be recorded during an initial *training run* and then used in subsequent *deployment runs* by specifying the switch `-XX:SharedArchiveFile=<dynamic archive>`.

Leyden premain seeks to take this further, by using training runs to capture much more metadata and code for reuse in deployment runs.

In general, a training run is considered to be a representative execution of an application, with typical inputs and config, which runs startup through expected paths and states and warms up to a steady state.

This will work best on systems that handle a lot of similar, repetitive tasks, leading to stable peak performance. Not all systems are like this, of course.

During training, the JVM gathers initial states, profiles, and JIT code and produces a log (or CDS archive). Optionally, multiple training runs are executed, and resulting logs of data are merged. The application is then *distilled* (essentially by applying a condenser) into the optimized version.

NOTE

One interesting long-term possibility is to auto-train and hide some or all of the training steps “under the hood”—but there’s a lot of shorter-term work needed before this becomes possible.

Executing the optimized application is called a deployment run. The deployment run starts with initial states and benefits from archived profiles and code.

In general, the startup phase of an application resolves symbols, runs class init methods (`<clinit>`), and runs `invokedynamic` BSMs (e.g., for lambdas). This work can be performed in a training run and saved for replay in deployment, along with some initialization states and code.

The code can be reused from the various tiers of HotSpot’s tiered compiler, including C1 (which is a “conservative” JIT that does not make speculative optimizations and, thus, never needs to de-optimize) and the optimized code from C2 (i.e., Tier 4). See [Chapter 6](#) for more details on HotSpot’s JIT compilers and tiering.

The C1-compiled code can be used in place of interpreted code, which improves startup by avoiding both online recompilation and the interpreter. This helps particularly in the case of non-hot code paths that may never be compiled or that are encountered at startup but not after that. Initial performance results indicate that these time savings are significant.

It is also the intent that JIT code can be regenerated during startup from persisted profiles, if necessary.

At a high level, training runs (which observe the app) can be seen as the dynamic flip side of static app analysis—or alternatively, a second-order form of profile-guided optimization.

The dynamic observations can be used as if they were statically deduced, provided we retain the possibility of de-optimization. Once captured, such data “looks static,” but it was “born dynamic,” and it can change,

triggering re-optimization. This combination of speculative techniques with “escape hatches” allowing for unplanned future events is a core competency of HotSpot.

In fact, there are very practical reasons why this approach is superior to total AOT compilation (as found in e.g., GraalVM Native Image, etc.).

For example, it is relatively common for workloads to have “unusual days.”

In the financial industry, examples could be the U.S. non-farm payroll (NFP) dates or the option maturity dates (once per quarter).

On these unusual dates, a fully AOT-compiled system is likely to perform much worse than one with a dynamic VM still in the loop. This is because the fully static AOT version cannot back out the assumptions about code path execution that were derived from training runs, whereas Leyden could de-optimize and recompile.

At the time of writing (August 2024), the status of premain work is:

- Premain activities are derived automatically from training runs.
- Optimizable states generated for premain are dumped into the archive.

In the future, it is anticipated that user-defined activities could participate as well. However, this will require work on characterizing things such as which user code is trusted as pure (e.g., via such things as new purity annotations).

This is to be expected—Leyden is an evolving technology that is still relatively early-stage.

Let’s move on to meet another long-term project within OpenJDK—Valhalla.

Valhalla

Project Valhalla is a long-running project that seeks to reorder the JVM at a very deep level.

In detail, the major goals of the project are:

- Aligning JVM memory layout behavior with the cost model of modern hardware.

- Extending generics to allow abstraction over all types, including primitives, values, and even `void`.
- Enabling existing libraries, especially the JDK, to compatibly evolve to fully take advantage of these features.

Buried within this description is a hint of one of the most complex efforts within the project: exploring the possibility of *value classes* within the JVM.

WARNING

Valhalla was launched in 2014, and over the last 10 years, the implementation design has changed significantly several times. Be very careful when reading about Valhalla that the information is up to date. For example, the description given in the first edition of this book is now completely wrong.

Recall that, up to and including version 21, Java has had only two types of values: primitive types and object references. To put this another way, the Java environment deliberately does not provide low-level control over memory layout.

To be a venue to explore and incubate advanced Java VM and language feature candidates.

—[Project Valhalla](#)

As a special case, this means that Java has no such thing as structs, and any composite data type can only be accessed by reference.

To understand the consequences of this, let's look at the memory layout of arrays. In [Figure 15-1](#) we can see an array of primitive `int`s. As these values are not objects, they are laid out at adjacent memory locations.

`int[]`



Figure 15-1. Array of ints

By contrast, the boxed integer is an object and so is handled by reference. This means that an array of `Integer` objects will be an array of references. This is shown in [Figure 15-2](#).

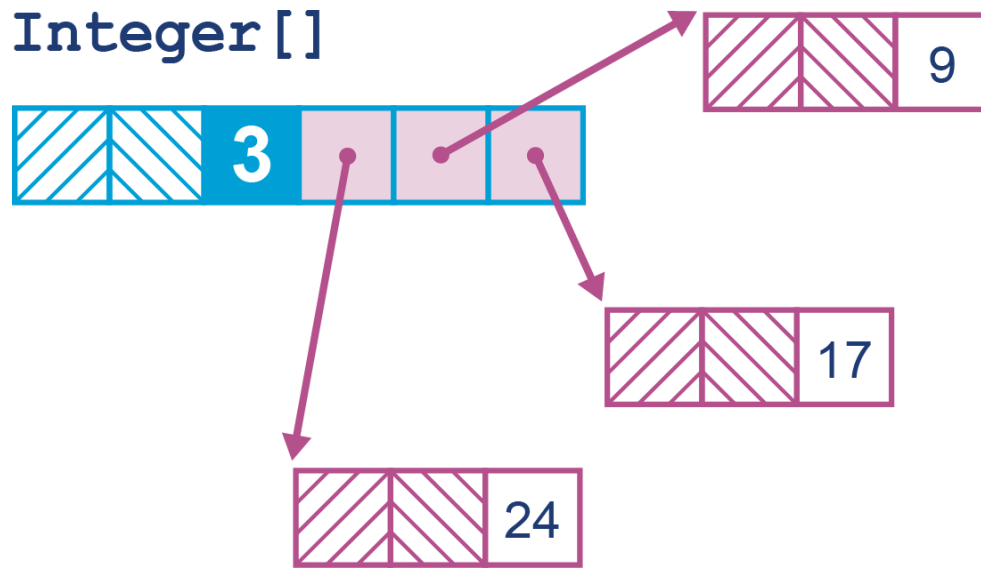


Figure 15-2. Array of integers

For over 25 years, this memory layout pattern has been the way that the Java platform has functioned. It has the advantage of simplicity but has a performance tradeoff—dealing with arrays of objects involves unavoidable indirections and attendant cache misses.

As a result, many performance-oriented programmers would like the ability to define types that can be laid out in memory more effectively. This would also include removing the overhead of needing a full object header for each item of composite data.

For example, a point in three-dimensional space, a `Point3D`, really only comprises the three spatial coordinates. As of Java 21, such a type can be represented as an object type with three fields:

```
public record Point3D(double x, double y, double z) {}
```

Therefore, an array of points will have the memory layout shown in [Figure 15-3](#).

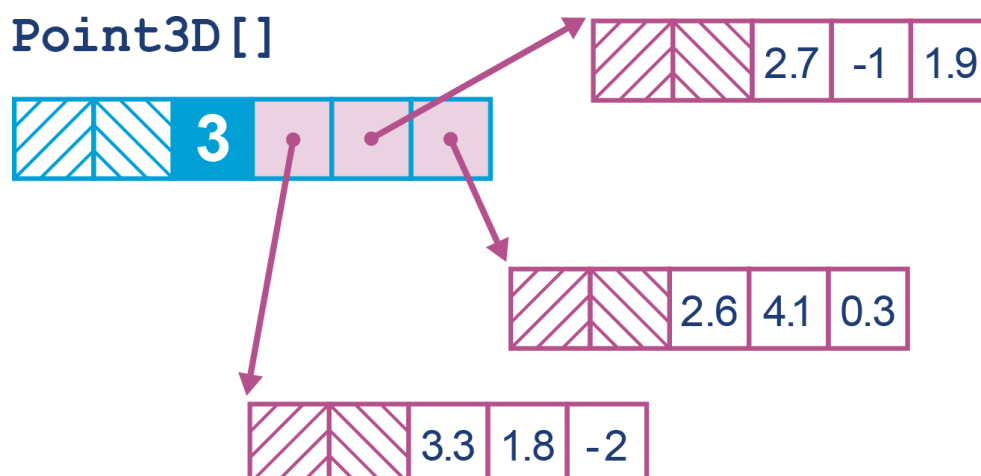


Figure 15-3. Array of Point3Ds

When this array is being processed, each entry must be accessed via an additional indirection to get the coordinates of each point. This has the potential to cause a cache miss for each point in the array, for no real benefit.

It is also the case that object identity is meaningless for the `Point3D` types. This means they are equal if and only if all their fields are equal. This is broadly what is meant by a *value class* in the Java ecosystem.

If this concept can be implemented in the JVM, then for simple types such as spatial points, a memory layout such as that shown in [Figure 15-4](#) could be far more efficient.

Point3D[] (value type)



Figure 15-4. Array of “struct-like” `Point3Ds`

Iterating over an array of these nested points is now much more efficient because of memory and cache locality, as well as saving the cost of the headers of the individual objects. Not only this, but then other possibilities (such as user-defined types that behave in a similar way to built-in primitive types) also emerge.

TIP

With “struct-like” arrays, there is the potential to call foreign functions using Project Panama. One example would be to offload the struct to a GPU for a vector processing operation for faster and less power-intensive processing.

However, there are some key conceptual difficulties in this area. One important problem is related to the original design decisions made in the early days of Java. This is the fact that the Java type system lacks a *top type*, so there is no type that is the supertype of both `Object` and `int`. We can also say that the Java type system is not *single-rooted*.

As a consequence, when generics were added way back in Java 5, it was decided that type variables could range only over reference types (subtypes of `Object`). Thus, there is no obvious way to construct a consistent meaning for, say, `List<int>`. Instead, Java uses *type erasure* to implement backward-compatible generic types over reference types.

Sometimes people complain about type erasure, but this mechanism is not responsible for the lack of a top type and the resulting lack of primitive collections.

If the Java platform is to be extended to include value types, then the question naturally arises whether value types can be used as type parameter values. If not, then this would seem to greatly limit their usefulness. Therefore, the design of value types has always included the assumption that they will be valid as values of type parameters in an enhanced form of generics.

Valhalla may be motivated by performance considerations, but a better way to view it is as enhancing abstraction, encapsulation, safety, expressiveness, and maintainability—without giving up performance.

—[Brian Goetz](#)

The current design of Valhalla strives to live up to the principle: “Codes like a class, works like an int.” There is just a single new keyword (`value`) to indicate that a class is a value class—all current classes are now understood to be *identity classes* (a concept that has not been needed until now).

The JVM bytecode also has only minor changes and does not currently require any new bytecodes to be defined.

One of the most obvious changes is in the implementation of value comparison (i.e., the `if_acmpeq` bytecode). In current versions of Java, this is just a bitwise comparison—two primitives are equal if they have the same bits, and two object references are equal if they point to the same memory location.

However, comparison of value objects is more complex—two value objects are the same if and only if all their fields have the same value. This can cause problems, because value classes can have fields that are also value classes.

For example:

```
public value record VR0(VR1 vr1) {}
public value record VR1(VR2 vr2) {}
public value record VR2(VR3 vr3) {}
// ... and so on
public value record VRN(int i) {}
```

Now, consider comparing two objects of type `VR0`. They will be equal if and only if the embedded instances of `VRN` hold the same `int` value, as we can see for the case `N=3`:

```
var vr0a = new VR0(new VR1(new VR2(new VR3(42))));
var vr0b = new VR0(new VR1(new VR2(new VR3(73))));
var vr0c = new VR0(new VR1(new VR2(new VR3(42))));

System.out.println(vr0a == vr0b);
System.out.println(vr0a == vr0c);
System.out.println(vr0b == vr0c);
```

which will output:

```
false
true
false
```

However, to ascertain this, the VM must recurse through the definitions of equality for all the intermediate types.

This means there is now the possibility of arbitrary-depth recursive behavior in `if_acmpeq`, and this has important potentially negative performance consequences.

Note that these types of dependency chains must terminate sometime—value classes are not allowed to have fields that would cause cyclic dependencies, as it would be impossible to know how much space was required to lay out an object of that type.

In terms of JIT compilation, the major impact is in the required support in the C2 compiler, basically to avoid allocations as much as possible and implement “fancy boxing” for value objects. In terms of handling the new equality semantics, there are cases where the JIT compiler can infer behavior, but also cases where it can’t.

Finally, these changes to the VM must be extremely carefully implemented. They must not, even in the worst possible case, cause performance regressions in existing code when Valhalla is not enabled.

At the time of writing (August 2024), it is unclear which release of Java will eventually introduce value types as a production feature.

Conclusions

In the new edition of this book, we have taken forward the material from the first edition that is still relevant to the modern Java developer. At the same time, we have introduced the techniques of cloud technology that are increasingly essential for applications that live in the cloud.

It is no longer enough for a performance-conscious Java engineer to have a journeyman's knowledge of the JVM's execution model and GC. New, cloud native techniques such as orchestration and observability are now part of the daily work of many—perhaps even most—Java developers and operations staff.

At the same time, the fundamentals of software performance engineering (in any environment) have not changed and still require the same knowledge and diligent application. Engineers now have more layers and more complex multivariate optimization problems to solve day to day. The body of knowledge also continues to grow, resulting in more specialization and coordination among engineers.

We hope that you will find the information and techniques we have presented in this book useful. It is intended as the starting point for your own unique performance journey rather than a complete guide. Good luck!